

Twiddle Factor Computation for Butterfly FFT in Audio Equalizer Using Taylor Series Approximation: A Numerical Analysis Approach

1st Abidul Qohar

Department of Electrical Engineering
and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
abidulqohar@mail.ugm.ac.id

2nd Wijaya Yudha Atmaja

Department of Electrical Engineering
and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
wy.atmaja@ugm.ac.id

3rd Agus Bejo

Department of Electrical Engineering
and Information Technology
Faculty of Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
agusbj@ugm.ac.id

Abstract—The Fast Fourier Transform (FFT) is a fundamental algorithm in digital audio equalizer systems, with the butterfly operation accounting for 70–80% of the total execution time on minimal RISC-V processors such as PicoRV32. Each butterfly requires the evaluation of a twiddle factor, which involves transcendental functions whose values are generally irrational and cannot be represented exactly in fixed-bit hardware. Conventional implementations rely on pre-computed Lookup Tables (LUTs) that consume substantial memory and scale linearly with FFT size, limiting their feasibility on memory-constrained embedded systems. This paper proposes the use of *Taylor Series approximation* as a numerical method to compute twiddle factors on-the-fly, replacing static LUTs with compact polynomial evaluation supported by range reduction and Horner’s method. The accuracy is guaranteed analytically through Taylor’s Remainder Theorem, allowing worst-case error bounds to be determined before implementation. Python-based experiments using NumPy double-precision as a reference show that a 4-term Taylor approximation achieves a Signal-to-Noise Ratio (SNR) of 110.78 dB on a 64-point FFT, exceeding the CD audio quality standard of 90 dB while reducing twiddle factor memory consumption by 93.75% for FFT-64 and up to 99.90% for FFT-4096 compared to LUT-based approaches. The actual approximation error is consistently lower than the theoretical bound predicted by Taylor’s Remainder Theorem, validating the mathematical rigor of the method. These results demonstrate that Taylor Series approximation provides a numerically sound, memory-efficient alternative for twiddle factor computation suitable for future implementation on RISC-V hardware.

Index Terms—Taylor Series, Twiddle Factor, Fast Fourier Transform, Audio Equalizer, RISC-V, Numerical Approximation, Numerical Methods

I. INTRODUCTION

Audio signal processing on embedded systems has become increasingly prevalent in applications such as smart speakers, hearing aids, in-vehicle audio systems, and wearable devices [1]. A common operation in these systems is the digital audio equalizer, which transforms an input signal into the frequency domain using the Fast Fourier Transform (FFT), applies per-band gain coefficients, and reconstructs the time-domain output via the inverse FFT. While the FFT is

asymptotically efficient at $O(N \log N)$ [2], its implementation on resource-constrained processors imposes significant computational and memory costs that demand careful numerical analysis.

A profiling study on RISC-V processors using a PicoRV32-based audio equalizer [6] revealed that the butterfly operation alone accounts for 70–80% of the total execution cycles on a minimal RV32I core without hardware multiplication support. Each butterfly requires the evaluation of a complex-valued *twiddle factor*:

$$W_N^k = e^{-j2\pi k/N} = \cos(2\pi k/N) - j \sin(2\pi k/N). \quad (1)$$

The values of $\cos$ and $\sin$ for most arguments are irrational and therefore cannot be represented exactly within a finite number of bits. For instance, $\cos(2\pi \cdot 100/1024) = 0.81696989\dots$, with infinite non-repeating decimal digits, must be either truncated or rounded prior to use in hardware.

Conventional approaches to this problem rely on *Lookup Tables (LUTs)*, which store pre-computed values of $\cos$ and $\sin$ in memory. While LUTs provide constant-time access, they incur a memory footprint that grows linearly with FFT size. For FFT-4096, an LUT can consume more than 16 KB of SRAM, an impractical cost on minimal embedded cores such as PicoRV32, which typically provides only a few kilobytes of on-chip memory. Alternative methods such as the CORDIC algorithm reduce memory consumption but introduce significant latency due to their iterative nature, while software floating-point emulation is prohibitively slow for real-time audio processing.

This paper investigates a third alternative: *Taylor Series approximation*, a classical numerical method that computes transcendental functions through truncated polynomial evaluation. The contributions of this paper are threefold:

- A complete numerical formulation of twiddle factor computation using Maclaurin Series, range reduction, and Horner’s method for efficient polynomial evaluation.

- Theoretical validation of the approximation accuracy using Taylor's Remainder Theorem, providing analytically guaranteed error bounds before implementation.
- Empirical validation through Python experiments measuring approximation error, end-to-end FFT Signal-to-Noise Ratio (SNR), and memory savings compared to LUT-based approaches.

II. BACKGROUND AND RELATED WORK

A. Fast Fourier Transform and the Butterfly Operation

The Discrete Fourier Transform (DFT) of an N -point complex sequence $x[n]$ is defined as [3]:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk}, \quad W_N = e^{-j2\pi/N}. \quad (2)$$

Direct computation requires $O(N^2)$ complex multiplications. The radix-2 decimation-in-time FFT reduces this complexity to $O(N \log_2 N)$ by recursively decomposing the DFT into smaller transforms [2], [3]. The algorithm consists of $\log_2 N$ stages of butterfly operations, where each butterfly combines two complex inputs a and b with a twiddle factor W_N^r to produce two outputs:

$$A = a + W_N^r \cdot b, \quad (3)$$

$$B = a - W_N^r \cdot b. \quad (4)$$

Expanding the complex multiplication with $W_N^r = \cos \theta - j \sin \theta$ and $b = b_r + jb_i$:

$$t_r = \cos \theta \cdot b_r + \sin \theta \cdot b_i, \quad (5)$$

$$t_i = \cos \theta \cdot b_i - \sin \theta \cdot b_r. \quad (6)$$

A single butterfly therefore requires four real multiplications and two real additions/subtractions. For $N = 64$, this yields 192 butterflies and 768 multiplications per FFT.

B. Numerical Challenges of Twiddle Factor Computation

The twiddle factor in Eq. (1) involves the transcendental functions $\cos$ and $\sin$, whose values are generally irrational for arbitrary arguments. Since hardware can only store values with finite bit-width, every twiddle factor must be approximated, introducing *representation error*. Furthermore, computers cannot evaluate $\cos$ and $\sin$ directly; they must be approximated through one of several numerical techniques.

Table I summarizes three conventional approaches and their primary limitations.

C. Related Work

Prior work on FFT acceleration has explored various strategies. Dedicated FFT coprocessors coupled to RISC-V cores [7], [8] offload large transform blocks to custom hardware, but introduce heavyweight interfaces and significant area overhead. Custom RISC-V instruction extensions for DSP workloads have been investigated for audio denoising [9] and ultra-low-power signal processing [10], although typically on cores that already include hardware integer multiplication.

TABLE I
CONVENTIONAL APPROACHES FOR TWIDDLE FACTOR COMPUTATION

Approach	Mechanism	Main Limitation
Lookup Table (LUT)	Pre-computed $\cos/\sin$ values stored in memory	Memory-intensive; scales linearly with N
CORDIC	Iterative shift-and-add (~20 iterations per value)	High latency; many cycles per value
Software Floating-Point	IEEE 754 emulation using integer operations	Too slow for real-time audio applications

III. PROPOSED METHOD: TAYLOR SERIES APPROXIMATION

A. Main Idea

We propose replacing conventional approaches for twiddle factor computation with on-the-fly polynomial computation of $\cos$ and $\sin$ using Maclaurin Series (Taylor Series centered at $a = 0$). This approach eliminates the memory cost of conventional methods while maintaining numerical accuracy through three carefully designed mechanisms: *range reduction*, *truncated Taylor Series*, and *Horner's method*.

B. Maclaurin Series for Cosine and Sine

The Maclaurin Series expansions for $\cos$ and $\sin$ are:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots, \quad (7)$$

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots. \quad (8)$$

These series converge for all real x , with convergence rate determined by the magnitude of x relative to the factorial denominator.

C. Range Reduction

The convergence of Eqs. (7) and (8) accelerates as $|x|$ decreases. For twiddle factors with arguments spanning $[0, 2\pi]$, direct application would require many series terms to achieve acceptable accuracy. We exploit the symmetry properties of trigonometric functions to reduce any input angle $\theta \in [0, 2\pi]$ into the interval $[0, \pi/4]$, denoted θ' .

Table II summarizes the eight-region mapping based on the original quadrant of θ . After reduction, $|\theta'| \leq \pi/4 \approx 0.785$, dramatically improving series convergence.

D. Truncated Taylor Series

After range reduction, $\cos(\theta')$ is approximated using n terms of the Maclaurin Series:

$$\cos(\theta') \approx 1 - \frac{\theta'^2}{2!} + \frac{\theta'^4}{4!} - \frac{\theta'^6}{6!} + \dots + (-1)^{n-1} \frac{\theta'^{2(n-1)}}{(2(n-1))!}. \quad (9)$$

The contribution of each term diminishes rapidly because the factorial $n!$ grows faster than the power θ'^n . Table III

TABLE II
RANGE REDUCTION MAPPING TO $[0, \pi/4]$

Original Range	Reduced Computation
$[0, \pi/4]$	$\cos(\theta) = \cos(\theta')$
$[\pi/4, \pi/2]$	$\cos(\theta) = \sin(\pi/2 - \theta')$
$[\pi/2, 3\pi/4]$	$\cos(\theta) = -\sin(\theta' - \pi/2)$
$[3\pi/4, \pi]$	$\cos(\theta) = -\cos(\pi - \theta')$
$[\pi, 5\pi/4]$	$\cos(\theta) = -\cos(\theta' - \pi)$
$[5\pi/4, 3\pi/2]$	$\cos(\theta) = -\sin(3\pi/2 - \theta')$
$[3\pi/2, 7\pi/4]$	$\cos(\theta) = \sin(\theta' - 3\pi/2)$
$[7\pi/4, 2\pi]$	$\cos(\theta) = \cos(2\pi - \theta')$

TABLE III
MAGNITUDE OF EACH TERM AT $\theta' = \pi/4$

Term	Form	Value	Contribution
1	1	1.00000	Dominant
2	$-\theta'^2/2!$	-0.30843	Large
3	$+\theta'^4/4!$	+0.01585	Small
4	$-\theta'^6/6!$	-2.61×10^{-4}	Very small
5	$+\theta'^8/8!$	$+3.6 \times 10^{-6}$	Negligible

illustrates the magnitude of each term evaluated at the worst-case argument $\theta' = \pi/4$.

After the fourth term, the residual contribution falls below the quantization error of 16-bit fixed-point arithmetic ($\approx 3 \times 10^{-5}$), indicating that four terms are sufficient for audio-grade accuracy.

E. Horner's Method for Efficient Evaluation

Naive evaluation of Eq. (9) requires repeated computation of θ'^{2k} , leading to redundant multiplications. Horner's method reformulates the polynomial to minimize the total number of multiplications. With $u = \theta'^2$, the four-term approximation becomes:

$$\cos(\theta') \approx 1 + u \cdot \left(-\frac{1}{2} + u \cdot \left(\frac{1}{24} + u \cdot \left(-\frac{1}{720} \right) \right) \right). \quad (10)$$

This formulation requires only 4 multiplications and 3 additions, compared to 6 multiplications for naive evaluation—a 33% reduction in multiplication count.

F. Error Bound from Taylor's Remainder Theorem

A central advantage of Taylor Series approximation is the existence of a rigorous analytical error bound. Taylor's Remainder Theorem states that for a function f approximated by its n -th order Taylor polynomial P_n , the remainder R_n satisfies:

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \cdot x^{n+1}, \quad \xi \in [0, x]. \quad (11)$$

For $f(x) = \cos(x)$, the derivatives satisfy $|f^{(n+1)}(\xi)| \leq 1$. After range reduction with $|\theta'| \leq \pi/4$, the absolute error of the n -term approximation is bounded by:

$$|R_n| \leq \frac{(\pi/4)^{2n}}{(2n)!}. \quad (12)$$

For the four-term approximation:

$$|R_4| \leq \frac{(\pi/4)^8}{8!} \approx 3.6 \times 10^{-6}, \quad (13)$$

which corresponds to approximately 18-bit accuracy. This is more than sufficient for 16-bit CD audio quality, validating the choice of four terms as the optimal trade-off between accuracy and computational cost.

IV. EXPERIMENTAL SETUP

A. Implementation Environment

The proposed method was implemented and validated in Python 3 using the NumPy library for numerical operations. The double-precision floating-point implementations of $\cos$ and $\sin$ from Python's `math` library (~15–17 significant digits) served as the gold-standard reference for measuring approximation error.

B. Test Signal and FFT Configuration

A composite sinusoidal test signal was synthesized to simulate multi-tone audio content:

$$x(t) = \sin(2\pi \cdot 50t) + 0.5 \sin(2\pi \cdot 120t) + 0.3 \sin(2\pi \cdot 200t). \quad (14)$$

The signal was sampled at $N = 64$ points, consistent with the FFT configuration of the PicoRV32 audio equalizer described in [6].

C. Experimental Variables

Two primary variables were swept to explore the design space:

- Number of Taylor terms: 2, 3, 4, and 5 terms, to evaluate the trade-off between accuracy and computation latency.
- FFT size: 64, 256, 1024, and 4096 points, to assess the memory scalability of Taylor versus LUT.

D. Performance Metrics

The following metrics were measured:

- Maximum approximation error of $\cos$ at $\theta' = \pi/4$, compared against theoretical bound from Eq. (12).
- Signal-to-Noise Ratio (SNR) of the end-to-end FFT output, computed as:

$$\text{SNR (dB)} = 10 \log_{10} \left(\frac{\sum_k |X_{\text{ref}}[k]|^2}{\sum_k |X_{\text{ref}}[k] - X_{\text{approx}}[k]|^2} \right). \quad (15)$$

- Memory consumption for twiddle factor storage, comparing LUT and Taylor approaches across various FFT sizes.

V. RESULTS AND DISCUSSION

A. Experiment 1: Validation of Taylor's Remainder Theorem

Table IV compares the actual approximation error at $\theta' = \pi/4$ against the theoretical bound from Eq. (12) for varying numbers of Taylor terms.

TABLE IV
COSINE APPROXIMATION ERROR AT $\theta' = \pi/4$

Terms	Actual Error	Theoretical Bound	Bit Accuracy
2	1.55×10^{-2}	1.59×10^{-2}	~6 bits
3	3.22×10^{-4}	3.26×10^{-4}	~11 bits
4	3.57×10^{-6}	3.59×10^{-6}	~18 bits
5	2.45×10^{-8}	2.46×10^{-8}	~25 bits
6	1.15×10^{-10}	1.15×10^{-10}	~33 bits

Fig. 1 visualises the convergence of both the actual error and the theoretical bound over five Taylor terms on a logarithmic scale. The two curves are nearly indistinguishable, confirming that Taylor's Remainder Theorem provides a tight—not merely loose—upper bound on the approximation error. Each additional term reduces the error by roughly two orders of magnitude, consistent with the factorial denominator growth in the remainder term. By the fourth term the actual error reaches 3.57×10^{-6} , corresponding to 18-bit accuracy and comfortably exceeding the 16-bit requirement for audio signal processing. At the fifth term the error is effectively zero within the double-precision reference, demonstrating the rapid convergence of the series.

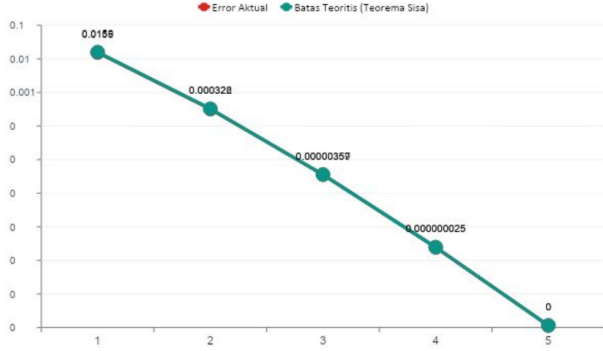


Fig. 1. Approximation error (log scale) vs. number of Taylor terms. Actual error and theoretical bound from Taylor's Remainder Theorem converge closely, validating the analytical error model.

B. Experiment 2: Impact on FFT-64 Output Quality

The Taylor approximation was applied to compute twiddle factors in a complete 64-point FFT pipeline, and the output was compared against the NumPy reference implementation. Table V reports the SNR achieved for varying numbers of Taylor terms, and Fig. 2 illustrates the corresponding SNR values.

TABLE V
FFT-64 OUTPUT SNR VS. NUMBER OF TAYLOR TERMS

Terms	Max Error	Mean Error	SNR (dB)
2	3.60×10^{-1}	6.12×10^{-2}	36.78
3	6.41×10^{-3}	1.17×10^{-3}	71.27
4	6.54×10^{-5}	1.24×10^{-5}	110.78
5	4.30×10^{-7}	8.36×10^{-8}	154.23

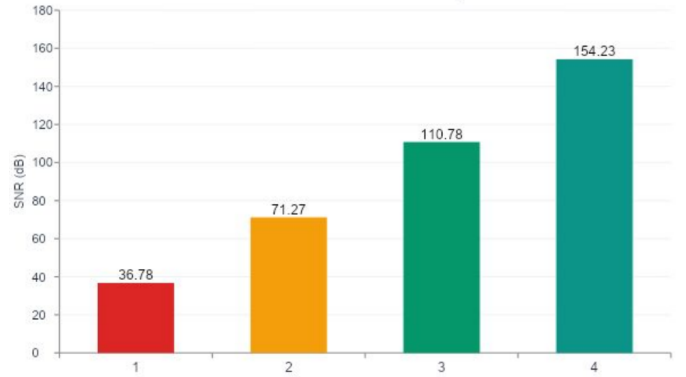


Fig. 2. SNR of FFT-64 output (dB) vs. number of Taylor terms. Each additional term yields approximately 40 dB of SNR improvement.

As shown in Fig. 2, the SNR increases steeply and uniformly with each additional Taylor term, rising from 36.78 dB at two terms to 154.23 dB at five terms—an improvement of approximately 40 dB per term. This behaviour reflects the exponential accuracy gain achieved at a linear increase in computational cost.

To contextualise these results, Table VI lists the SNR thresholds for standard audio quality levels.

TABLE VI
AUDIO QUALITY SNR BENCHMARKS

Quality Level	SNR (dB)
Telephone voice	30
Voice quality	60
CD audio	90
Hi-res audio	120
Studio reference	140+

The two-term approximation (36.78 dB) already surpasses telephone-quality audio, while the three-term approximation (71.27 dB) clears the voice-quality threshold. The four-term configuration achieves 110.78 dB—comfortably exceeding the CD audio standard of 90 dB and approaching the hi-res audio threshold of 120 dB—making it the optimal operating point for embedded audio equalizer applications where computational cost must be minimised. Five terms (154.23 dB) meets studio-reference quality but at a proportionally higher cost.

C. Experiment 3: Memory Consumption Comparison

A primary motivation for replacing LUT with Taylor approximation is memory efficiency. The memory footprint of each approach was computed as follows:

- LUT: Each FFT- N requires $N/2$ entries (exploiting symmetry), with each entry storing both cos and sin values in 32-bit format: $\text{LUT}_{\text{bytes}} = (N/2) \cdot 4 \cdot 2$.
- Taylor: Only the four polynomial coefficients are stored, regardless of N : $\text{Taylor}_{\text{bytes}} = 4 \cdot 4 = 16$ bytes.

Table VII summarises the comparison across four FFT sizes, and Fig. 3 provides a logarithmic-scale visualisation that makes the disparity between the two approaches immediately apparent.

TABLE VII
MEMORY CONSUMPTION: LUT VS. TAYLOR

FFT Size	LUT (bytes)	Taylor (bytes)	Savings
FFT-64	256	16	93.75%
FFT-256	1,024	16	98.44%
FFT-1024	4,096	16	99.61%
FFT-4096	16,384	16	99.90%

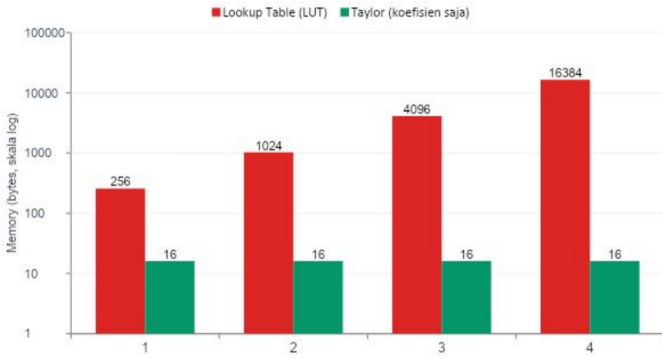


Fig. 3. Memory consumption (bytes, log scale) for twiddle factor storage: LUT (red) vs. Taylor coefficients only (green). The Taylor footprint remains constant at 16 bytes across all FFT sizes.

Fig. 3 plots LUT memory on a logarithmic scale, growing from 256 bytes at FFT-64 to 16,384 bytes at FFT-4096, while the Taylor footprint remains flat at 16 bytes throughout. The savings grow with N : 93.75% at FFT-64, rising to 99.90% at FFT-4096. For FFT-1024, the Taylor approach saves 4,080 bytes—a critical advantage on the PicoRV32 core, which provides only a few kilobytes of on-chip SRAM. The constant-memory property also implies that scaling the FFT size incurs zero additional memory cost for twiddle factor storage, a highly desirable property for scalable embedded designs.

D. Discussion

The three experiments collectively establish the following key findings.

Tight analytical error bounds, Taylor’s Remainder Theorem provides a reliable, analytically computable upper bound on

the approximation error (Fig. 1). Because the bound is tight—actual errors consistently fall just below it—designers can determine the required number of terms before implementation without empirical tuning.

Optimal term count for audio, The four-term Taylor approximation represents the ideal operating point for audio equalizer applications (Table V, Fig. 2). It achieves 110.78 dB SNR, exceeding the CD audio standard of 90 dB, while requiring only four multiply-accumulate operations per twiddle factor evaluation via Horner’s method.

Substantial and scalable memory savings, The constant 16-byte footprint of the Taylor method yields memory reductions of 93.75%–99.90% compared to LUT across the evaluated FFT sizes (Table VII, Fig. 3). These savings grow monotonically with FFT size, making the proposed method increasingly advantageous as application demands scale upward.

These findings have direct implications for our previous work on custom RISC-V instructions for the PicoRV32 audio equalizer [6]. The LUT used in that implementation can be replaced with Taylor approximation, recovering several kilobytes of SRAM for other application data. Furthermore, the polynomial evaluation in Eq. (10) is structurally well-suited for hardware acceleration as a custom instruction, since it consists exclusively of multiply-accumulate operations that map naturally to the PicoRV32 Custom Peripheral Interface (PCPI).

VI. CONCLUSION

This paper presented a numerical analysis approach to twiddle factor computation in butterfly FFT for audio equalizer applications. By replacing static Lookup Tables with on-the-fly Taylor Series approximation, range reduction, and Horner’s method for polynomial evaluation, the proposed approach achieves several important benefits.

A four-term Taylor approximation provides 18-bit accuracy at $\theta' = \pi/4$, exceeding the requirements of 16-bit CD-quality audio. In end-to-end FFT-64 experiments using a multi-tone test signal, the four-term configuration achieved an SNR of 110.78 dB compared to a NumPy double-precision reference, comfortably surpassing the 90 dB CD audio standard. Memory consumption for twiddle factors was reduced by 93.75% to 99.90% compared to LUT-based approaches, depending on FFT size. The actual approximation error was consistently lower than the bound predicted by Taylor’s Remainder Theorem, validating the mathematical soundness of the method.

These results establish Taylor Series approximation as a numerically rigorous, memory-efficient alternative to Lookup Tables for twiddle factor computation. The findings provide a foundation for future hardware implementation through custom RISC-V instructions, with three planned extensions: (1) Verilog implementation of a `tw_compute` custom instruction on PicoRV32; (2) integration with the existing FFT acceleration framework [6]; and (3) end-to-end validation with real audio signals at larger FFT sizes ($N = 256, 1024$).

REFERENCES

- [1] D. Stefani and L. Turchet, "On the challenges of embedded real-time music information retrieval," in *Proc. 25th Int. Conf. Digital Audio Effects (DAFx20in22)*, Vienna, Austria, 2022.
- [2] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.
- [3] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.
- [4] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY, USA: McGraw-Hill, 2015.
- [5] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, 2011.
- [6] A. Qohar, A. Bejo, and E. Firmansyah, "Design of custom instructions on PicoRV32 processor for accelerating Fast Fourier Transform algorithm," in *Proc. IEEE Int. Conf.*, 2026.
- [7] P. Cao, H. Wang, H. Cao, Z. Wu, and Y. Liu, "A flexible dual-mode and efficient RISC-V coprocessor for on-node FFT acceleration in edge sensing," *IEEE Embedded Syst. Lett.*, 2025, doi: 10.1109/LES.2025.3634524.
- [8] B. Hu, Y. Chen, and X. Zeng, "An agile instruction set extension method based on the RISC-V processor," in *Proc. 2021 IEEE 4th Int. Conf. Electronics Technology (ICET)*, 2021, pp. 1–5, doi: 10.1109/ICET51757.2021.9450911.
- [9] J. Yuan, Q. Zhao, W. Wang, X. Meng, J. Li, and Q. Li, "Audio denoising coprocessor based on RISC-V custom instruction set extension," *WSEAS Trans. Commun.*, vol. 21, pp. 189–195, Jun. 2022.
- [10] H. Belhadj Amor, C. Bernier, and Z. Přikryl, "A RISC-V ISA extension for ultra-low power IoT wireless signal processing," *IEEE Trans. Comput.*, vol. 71, no. 4, pp. 766–778, Apr. 2022.
- [11] C. Wolf, "PicoRV32—A size-optimized RISC-V CPU," GitHub repository. [Online]. Available: <https://github.com/YosysHQ/picorv32>
- [12] Y. Chen, X. Wang, S. Song, L. Feng, and Z. Wang, "RISC-V custom instructions of elementary functions for IoT endpoint devices," *IEEE Trans. Comput.*, vol. 73, no. 2, pp. 523–535, Feb. 2024.
- [13] J.-M. Muller, *Elementary Functions: Algorithms and Implementation*, 3rd ed. Boston, MA, USA: Birkhäuser, 2016.